```
CHANNEL_FILE = "channel.txt"

def text_to_bin(t):
    return "".join(format(ord(c), "08b") for c in t)

def bin_to_text(b):
    return "".join(chr(int(b[i:i+8], 2)) for i in range(0, len(b), 8))

def encode_nibble(d):
    d1, d2, d3, d4 = (int(x) for x in d)
    p1, p2, p4 = d1 ^ d2 ^ d4, d1 ^ d3 ^ d4, d2 ^ d3 ^ d4
    return f"{p1}{p2}{d1}{p4}{d2}{d3}{d4}"

def sender(text):
    bits = text_to_bin(text)
    bits += "0" * ((-len(bits)) % 4)
    encoded = "".join(encode_nibble(bits[i:i+4]) for i in range(0, len(bits), 4))
    open(CHANNEL_FILE, "w").write(encoded)
    print("Sent:", encoded)

def check_block(block):
    b = [int(x) for x in block]
    c1 = b[0] ^ b[2] ^ b[4] ^ b[6]
    c2 = b[1] ^ b[2] ^ b[5] ^ b[6]
    c4 = b[3] ^ b[4] ^ b[5] ^ b[6]
    err = c1 + 2 * c2 + 4 * c4

    if err:
        b[err - 1] ^= 1

    return "".join(map(str, b)), err

def receiver():
    stream = open(CHANNEL_FILE).read().strip()
    data = ""

    for i in range(0, len(stream), 7):
        block, err = check_block(stream[i:i+7])
        print(f"Block {i//7+1}: {'error at bit '+str(err) if err else 'ok'}")
        data += block[2] + block[4] + block[5] + block[6]

    text = bin_to_text(data)
    print("Received:", text)
    return text
```

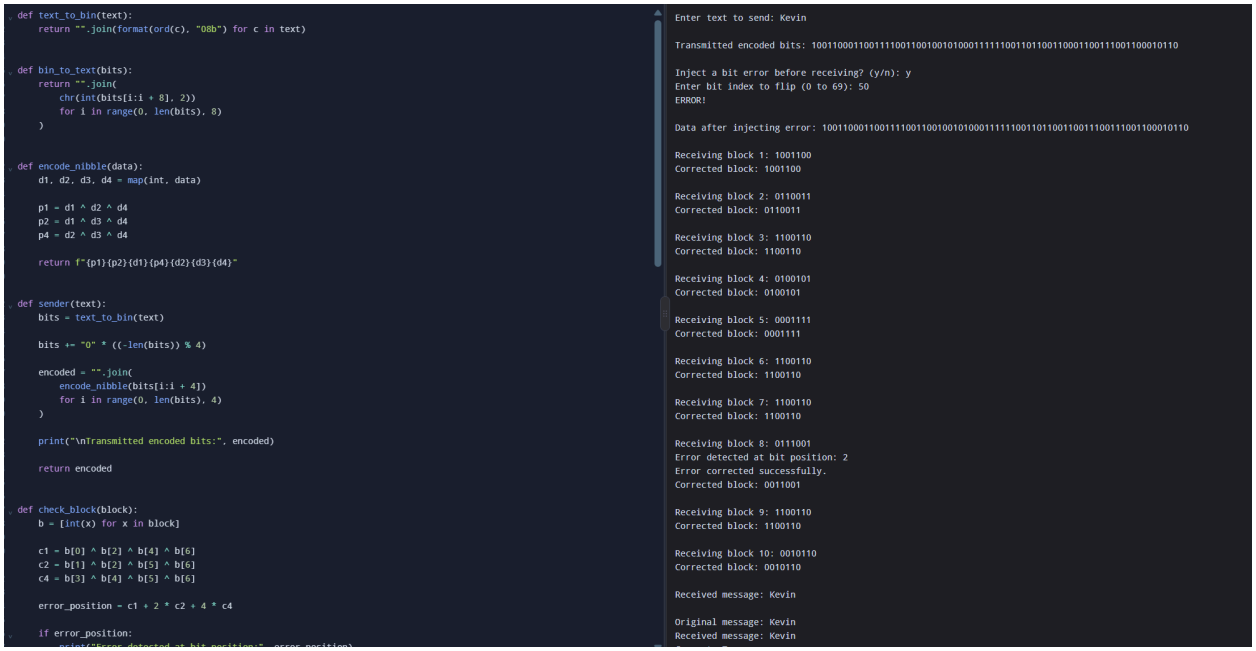
```
if __name__ == "__main__":
    msg = input("Enter text to send: ")
    sender(msg)

    choice = input("Inject a bit error before receiving? (y/n): ").strip().lower()

    if choice == "y":
        bits_len = len(open(CHANNEL_FILE).read().strip())
        pos = int(input(f"Enter bit index to flip (0 to {bits_len-1}): "))
        s = list(open(CHANNEL_FILE).read().strip())
        s[pos] = "1" if s[pos] == "0" else "0"
        open(CHANNEL_FILE, "w").write("".join(s))
        print(f"Flipped bit at index {pos}")

    out = receiver()
    print("Correct:", out == msg)
```

OUTPUT :



```
def text_to_bin(text):
    return "".join(format(ord(c), "08b") for c in text)


def bin_to_text(bits):
    return "".join(
        chr(int(bits[i:i + 8], 2))
        for i in range(0, len(bits), 8)
    )


def encode_nibble(data):
    d1, d2, d3, d4 = map(int, data)

    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p4 = d2 ^ d3 ^ d4

    return f"{p1}{p2}{d1}{p4}{d2}{d3}{d4}"


def sender(text):
    bits = text_to_bin(text)

    bits += "0" * ((-len(bits)) % 4)

    encoded = "".join(
        encode_nibble(bits[i:i + 4])
        for i in range(0, len(bits), 4)
    )

    print("\nTransmitted encoded bits:", encoded)

    return encoded


def check_block(block):
    b = [int(x) for x in block]

    c1 = b[0] ^ b[2] ^ b[4] ^ b[6]
    c2 = b[1] ^ b[2] ^ b[5] ^ b[6]
    c4 = b[3] ^ b[4] ^ b[5] ^ b[6]

    error_position = c1 + 2 * c2 + 4 * c4

    if error_position:
```

```
Enter text to send: Kevin

Transmitted encoded bits: 1001100011001111001100100101000111111001101100110001100111001100100110

Inject a bit error before receiving? (y/n): y
Enter bit index to flip (0 to 69): 50
ERROR!

Data after injecting error: 1001100011001111001100100101000111111001101100110001100111001100100110

Receiving block 1: 1001100
Corrected block: 1001100

Receiving block 2: 0110011
Corrected block: 0110011

Receiving block 3: 1100110
Corrected block: 1100110

Receiving block 4: 0100101
Corrected block: 0100101

Receiving block 5: 0001111
Corrected block: 0001111

Receiving block 6: 1100110
Corrected block: 1100110

Receiving block 7: 1100110
Corrected block: 1100110

Receiving block 8: 0111001
Error detected at bit position: 2
Error corrected successfully.
Corrected block: 0011001

Receiving block 9: 1100110
Corrected block: 1100110

Receiving block 10: 0010110
Corrected block: 0010110

Received message: Kevin

Original message: Kevin
Received message: Kevin
```